

GENAI IN TESTING

Accelerate quality without losing trust

Using GitHub Copilot to generate, review, and govern tests that hold up in production.

// 02 — WHO AM I

Moataz Nabil

ENGINEERING MANAGER / LEADER • AVIV GROUP — IMMOWELT

Building engineering organizations that ship better software, faster.

Platform Engineering

Developer Experience

Quality Engineering

moataznabil.blog • LinkedIn • GitHub

SHOW OF HANDS

How many of you have accepted
a Copilot test suggestion
without fully reading it?

A

Never

B

Once or twice

C

Regularly

D

It's my workflow

This is why we're here today.

The session flow

01	Intro & Setup Pipeline tour, Copilot activation, and the speed-vs-trust tension	15 min
02	Exercise A – Unit tests calculateDiscount(), weak vs strong prompts, 3 bugs exposed	25 min
03	Exercise B – Review AI tests Spot false confidence, fix weak assertions, expose 3 bugs – then make a flaky timing test deterministic	10 min
04	Exercise C – API tests Full checkout pipeline: Cart → Discount → Fraud → Payment → Notification	25 min
05	Exercise D – Component & E2E Tests StorePage with React Testing Library; Playwright checkout flow – hands-on test.fixme fill-in in checkout.spec.ts	20 min
06	Exercise E – CI guardrails Coverage gates, quality thresholds, copilot-instructions.md	10 min
07	Wrap-up & Q&A Trust Playbook, one action to take back, open discussion	15 min

Supplementary – Configuring Copilot for your team (instructions, prompt files, skills, agent mode, MCP): reference material, covered as time allows outside the core 120-min flow.

What you'll be able to do

You leave with reusable skills, not just ideas.

- 01 Generate production-grade unit, API, and integration tests with Copilot.
- 02 Spot and fix weak AI-generated tests before they reach main.
- 03 Apply trust guardrails: instructions, review checklists, and quality gates.
- 04 Integrate AI-assisted tests into CI/CD with context, skills, and MCP.

Prerequisites and setup

Have these ready so we spend the session building, not installing.

- ▶ A laptop with VS Code installed and updated
- ▶ Git installed and configured with your name and email
- ▶ The starter repository cloned and dependencies installed
- ▶ The GitHub Copilot extension, signed in and active
- ▶ A GitHub account with Copilot access enabled
- ▶ Comfort reading basic tests in your language of choice

WORKSHOP TOOLCHAIN

VS Code • Node.js 22+ • npm • Jest 30 • Supertest • React Testing Library • Playwright

REPOSITORY

github.com/moatazeldebsy/copilot-testing-workshop

It's a GitHub template – click Use this template for your own writable copy. No local setup? Open it in Codespaces via the green Code button.

AI is changing how code gets written

Developers now draft functions, refactor, and tests with an assistant in the loop. The output arrives faster than any team can review it line by line.

Testing and quality engineering feel that speed first – and absorb the risk when the guardrails are missing.

Faster

Test creation, scaffolding, and boilerplate

Riskier

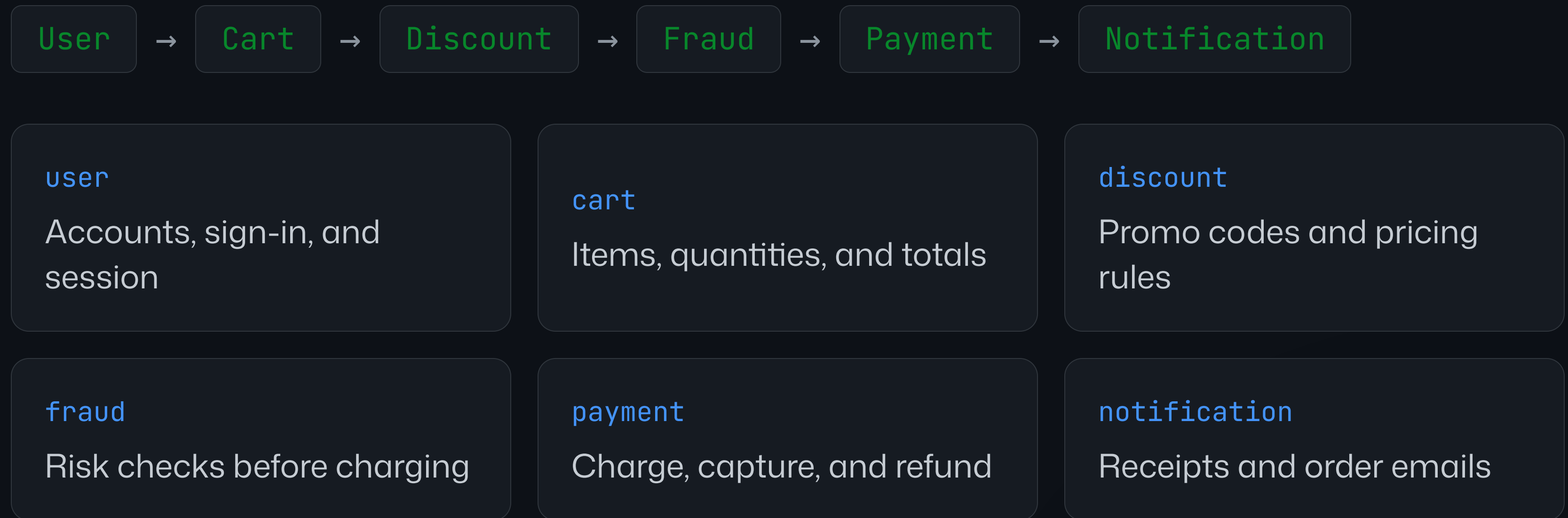
Flaky tests, false confidence, weak assertions

Reviewable

Only with clear standards and governance

The system we will test

One sample checkout system threads the whole workshop, so every test we generate has a real home.



A passing suite is not the same as a trustworthy one

Green

Tests that pass but assert almost nothing still report success.

Flaky

Generated tests can depend on timing, order, or live state.

False

Confidence rises while real coverage of behavior stays thin.

The goal of this workshop: keep the speed, and add the judgment that turns generated tests into trustworthy ones.

Where Copilot adds real value

+ Boilerplate and scaffolding

Setup, fixtures, and repetitive test structure

+ Mock and stub generation

Doubles for services, clients, and dependencies

+ Readability refactors

Clearer names, structure, and assertions

+ Edge-case suggestions

Nulls, empties, and boundaries you might miss

+ Test data fabrication

Realistic sample payloads and factories

+ Coverage of the obvious

Fast first drafts for well-understood logic

Where human judgment is essential

! What actually matters

Deciding which behaviors are worth testing

! Business rules and intent

Context the model cannot infer from code

! Edge cases that count

Separating real risks from noise

! Assertion quality

Verifying tests check the right outcome

! Risk and security calls

Threat scenarios and sensitive data handling

! Final accountability

A person owns what ships, not the assistant

PART 01

Generating tests with Copilot

Live patterns for unit, API, and integration tests – plus the prompts that produce useful first drafts.

// 13 EXERCISE A

Unit tests

Start from a pure function. Prompt Copilot for the happy path, then push it toward the boundaries.

- Ask for typical, empty, and invalid inputs
- Name each test for the behavior it proves
- Reject tests that assert nothing meaningful

```
calculateDiscount.test.ts

describe("calculateDiscount", () => {
  it("SAVE10 takes 10% off the subtotal", () => {
    const r = calculateDiscount(
      { subtotal: 100, code: "SAVE10" });
    expect(r.discountAmount).toBe(10);
    expect(r.finalTotal).toBe(90);
  });

  it("FLAT5 takes $5 off orders $20+", () => {
    const r = calculateDiscount(
      { subtotal: 50, code: "FLAT5" });
    expect(r.finalTotal).toBe(45);
  });

  it("returns zero for an unknown code", () => {
    const r = calculateDiscount(
      { subtotal: 100, code: "INVALID" });
    expect(r.discountAmount).toBe(0);
  });
});
```


Find and fix weak tests

Copilot accelerates; humans validate – make a passing test mean something.

Weak – passes, proves nothing

```
it("SAVE10 produces a discount", () => {  
  const r = calculateDiscount({ subtotal: 100, code: "SAVE10" });  
  
  // passes even if discount is $100  
  expect(r.discountAmount).toBeTruthy();  
});
```

Strong – asserts the behavior

```
it("deducts exactly 10%, not 100%", () => {  
  const r = calculateDiscount({ subtotal: 100, code: "SAVE10" });  
  expect(r.discountAmount).toBe(10); // BUG 1  
});  
  
it("FLAT5 skips below the $20 min", () => {  
  const r = calculateDiscount({ subtotal: 15, code: "FLAT5" });  
  expect(r.discountAmount).toBe(0); // BUG 2  
});  
  
it("finalTotal is never negative", () => {  
  const r = calculateDiscount({ subtotal: 3, code: "FLAT5" });  
  expect(r.finalTotal).toBeGreaterThanOrEqual(0); // BUG 3  
});
```

Prompt: "Improve this test – make assertions meaningful, add the missing edge cases." We didn't change the code, we changed the trust level.

The flaky-test moment

tests/unit/notificationService.test.ts

Weak assertions are one failure mode. Timing dependencies are another – and they are harder to spot because the test passes most of the time.

Flaky – depends on timing

```
it("sentAt within 50ms of the call", () => {
  const before = Date.now();
  const log = service.send(payload);
  const after = Date.now();

  // fails on a loaded CI runner
  expect(log.sentAt.getTime())
    .toBeLessThanOrEqual(after + 50);
});
```

Deterministic – mock the clock

```
it("sentAt is the mocked time", () => {
  jest.useFakeTimers().setSystemTime(
    new Date("2026-07-10T12:00:00Z"));

  const log = service.send(payload);

  expect(log.sentAt).toEqual(
    new Date("2026-07-10T12:00:00Z"));
  jest.useRealTimers();
});
```

Prompt: "Make this test deterministic." Mock the clock with `jest.useFakeTimers` – or assert structure (`sentAt` is a `Date`), not exact timing.

```
it("adds an item to the cart", async () => {
  const res = await request(app)
    .post(`/api/cart/${userId}/items`)
    .set("Authorization", `Bearer ${token}`)
    .send({ productId: "prod-1",
      price: 25, quantity: 1 });
  expect(res.status).toBe(201);
  expect(res.body.data.subtotal).toBe(25);
});

it("rejects a bad code with 400", async () => {
  const res = await request(app)
    .post("/api/discount/apply")
    .send({ code: "BOGUS", subtotal: 100 });
  expect(res.status).toBe(400);
  expect(res.body.error.code)
    .toBe("INVALID_DISCOUNT_CODE");
});

it("blocks a high-risk order", async () => {
  const res = await request(app)
    .post("/api/fraud/check")
    .send({ orderAmount: 10000,
      itemCount: 50, ipCountry: "XX" });
  expect(res.body.data.approved).toBe(false);
  expect(res.body.data.riskLevel).toBe("high");
});
```

// 16 EXERCISE C

API tests

Cover the contract: status codes, response shape, and the failure paths that protect your callers.

- Assert status and body, not just one
- Test validation and error responses
- Keep each case isolated and repeatable

// 17 - DEMO

Integration tests

Exercise real wiring across modules. Set up known state, act, and assert the system behaves end to end.

- Seed and tear down state per test
- Avoid hidden coupling between cases
- Watch for flakiness from shared resources

checkout.pipeline.test.ts

```
import {
  resetWorkshopData, cartService,
  discountService, fraudService,
  paymentService, notificationService,
} from '../src/app';

describe("checkout pipeline (integration)", () => {
  beforeEach(async () => {
    await resetWorkshopData();
  });

  it.todo('adds an item via cartService
    and reflects it in the cart total');
  it.todo('runs the full pipeline end-to-end
    and asserts the final order state');
});
```

These ship as `it.todo()` stubs on master. Optional, time-permitting demo – not part of the core 120-minute flow.

Component & E2E tests

Render the UI and drive it the way a user would. Assert what the person sees, not how the component is built.

- Query by role and visible text, not internals
- Mock fetch so the render stays deterministic
- E2E is hands-on: fill in `test.fixme` in `tests/e2e/checkout.spec.ts` – login, add-to-cart, discount, full checkout

```
StorePage.test.tsx

describe("StorePage", () => {
  beforeEach(() => {
    global.fetch = jest.fn();
  });

  it("renders products with Add buttons", async () => {
    mockFetchOnce({ data: { products: [
      { id: "prod_1",
        name: "Workshop T-Shirt",
        price: 25 } ] } });
    render(<StorePage {...props} />);
    await waitFor(() =>
      expect(screen.getAllByRole(
        "button", { name: /add/i }).length)
        .toBeGreaterThan(0));
  });

  it("shows item after clicking Add", async () => {
    const user = userEvent.setup();
    mockFetchOnce({ data: {
      items: [buildCartItem()] } });

    render(<StorePage {...props} />);
    await user.click(
      screen.getByTestId("add-prod_1"));

    await waitFor(() =>
      expect(screen.getByTestId("cart-items"))
        .toHaveTextContent("Workshop T-Shirt"));
  });
});
```

● ● ● factories.ts

```
export const buildCartItem = (over = {}) => ({
  id: "item-1",
  productId: "prod_1",
  name: "Workshop T-Shirt",
  price: 25,
  quantity: 1,
  ...over,
});

function mockFetchOnce(body, status = 200) {
  (fetch as jest.Mock)
    .mockResolvedValueOnce({
      ok: status < 300,
      status,
      json: async () => body,
    });
}
```

// 19 - TECHNIQUE

Test data and mocks

Use factories for readable defaults and mock external systems so tests stay fast and deterministic.

- Override only what each test cares about
- Mock network, time, and randomness
- Never put real secrets in fixtures

Refactoring for readability

Before

```
it("works", () => {  
  const r = calc(2, 3, "add");  
  expect(r).toBe(5);  
  const r2 = calc(5, 2, "sub");  
  expect(r2).toBe(3);  
});
```

After

```
it("adds two numbers", () => {  
  expect(calc(2, 3, "add")).toBe(5);  
});  
  
it("subtracts two numbers", () => {  
  expect(calc(5, 2, "sub")).toBe(3);  
});
```

One behavior per test, named for intent. When a test fails, the name alone should tell you what broke.

Prompt sheet

Don't improvise – paste these. Consistent prompts produce consistent tests across the whole team.

Unit tests

Generate Jest unit tests for this function.

Cover:

- valid input
- null / empty values
- boundary conditions
- invalid states

Use descriptive test names.
Avoid mocking unless necessary.

API tests

Generate Supertest API tests for this endpoint.

Include:

- 200 success
- 400 validation failure
- 401 unauthorized
- 500 internal error

Assert status AND body. Focus on business behavior, not internals.

Test review

Review these generated tests.

Identify:

- flaky assertions
- brittle mocks
- duplicate coverage
- weak assertions

Suggest concrete improvements.

PART 02

Configuring Copilot for your team

Instructions, prompt files, skills, agent mode, and MCP – the controls that make Copilot consistent, repeatable, and worth trusting.

How Copilot fits together

1 copilot-instructions.md

Repo-wide testing standards

- Jest
- Playwright
- no sleeps
- coverage $\geq$ 80%

2 AGENTS.md

Roles for autonomous work

- test agent
- API agent
- E2E agent
- scope & boundaries

3 skills/

On-demand playbooks

- pact-contracts
- flaky-test-hunt
- test-generation
- reusable templates

4 MCP

External tools & data

- browser
- database
- CI
- test reports

TESTING WORKFLOW

● AUTHOR

1

Write prompt



2

Load instructions



3

Select role/skill



● GENERATE

4

Use MCP tools



5

Generate tests



● VALIDATE

6

Run checks

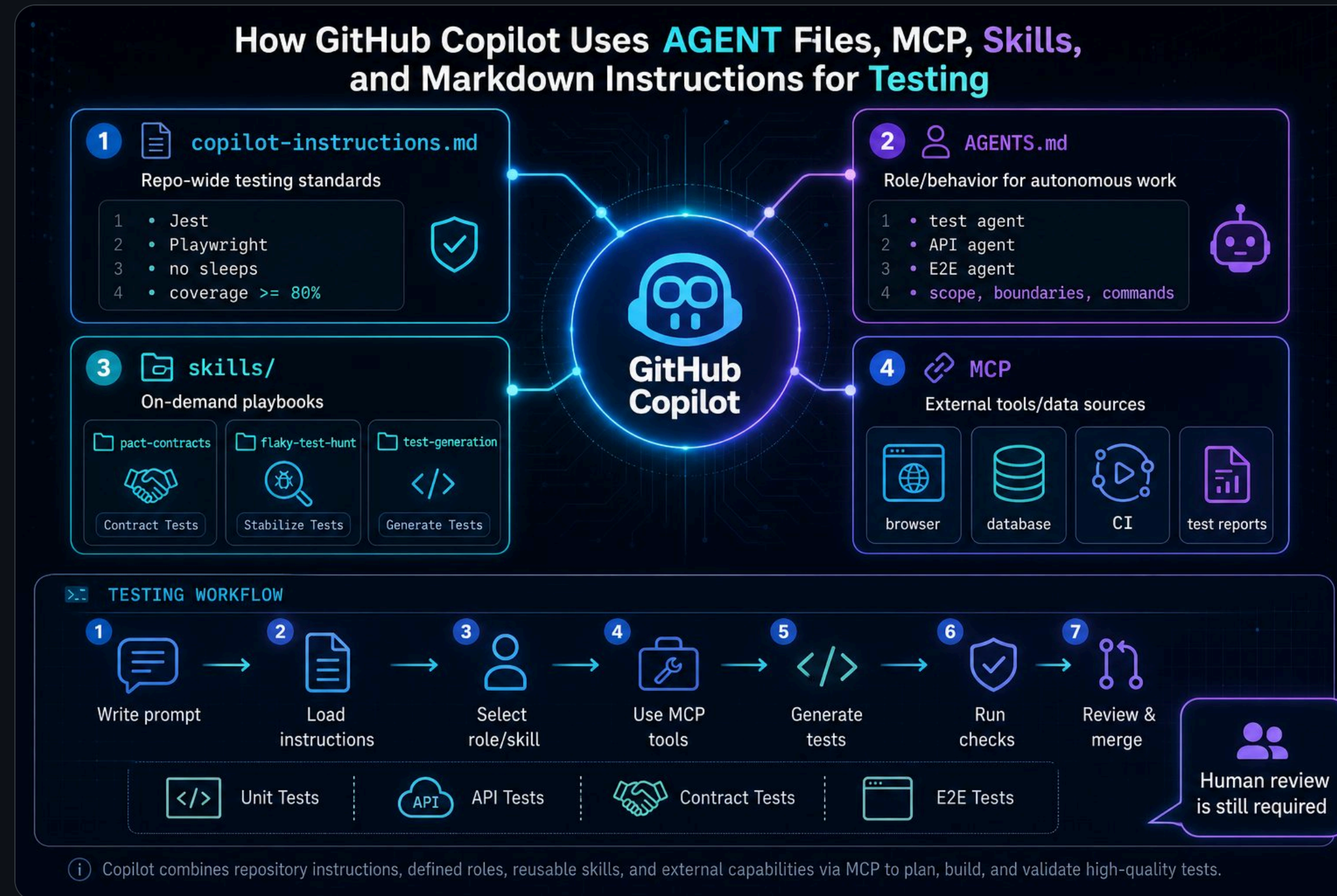


7

Review & merge

! Human review is still required

How it all connects



Custom instructions

Commit your standards as Markdown so Copilot follows them on every request – no more re-typing the same prompt.

- copilot-instructions.md – repo-wide rules
- *.instructions.md – path-scoped guidance
- AGENTS.md – shared across AI tools

workshop-exercises/.github/copilot-instructions.md

Testing standards

- AAA pattern; one behavior per it().
- Assert exact values – toBeTruthy() and toBeDefined() hide bugs.
- Cover golden path, boundaries, and error paths.
- Don't mock the module under test.
- No Math.random() or Date.now() in assertions.
- Validate status AND response envelope for APIs.
- Never include secrets or PII in fixtures.

Where config lives and the token strategy

EXPLORER · workshop-exercises

```
▶workshop-exercises/  
  AGENTS.md · copilot.md  
  .copilot/context/domain-rules.md  
  ▶.github/  
    copilot-instructions.md · workflows/test.yml  
    chatmodes/qa-reviewer.chatmode.md  
    prompts/generate-tests.prompt.md  
    ▶.github/skills/  
      ▶pact-contracts/  
      ▶flaky-test-hunt/  
      ▶test-generation/  
  ▶docs/  
    ai-testing-trust-playbook.md  
  ▶tests/  
    unit/ api/ components/ e2e/ integration/
```

TOKEN STRATEGY

Not everything loads at once – context is layered by cost and need.

Always loaded

TINY

AGENTS.md · copilot-instructions.md · critical root configs
Always in context

Role-scoped

MEDIUM

relevant docs · role-specific guidance · shared patterns
Loaded by role & task

On-demand

LARGE

skills/ playbooks · example tests · reference materials
Loaded when needed

Prompt files

Package a proven test-generation prompt as a file the whole team can invoke with a slash command.

- Live in `.github/prompts/*.prompt.md`
- Run as `/generate-tests` in chat
- Version-controlled and reviewed like code

```
generate-tests.prompt.md

---
mode: agent
description: Generate tests for a file
---

Write Jest tests for ${file}.

- Cover happy path, edge, and error cases.
- Reuse factories from tests/factories.
- Do not test private functions.
- Stop and ask if behavior is unclear.
```

Your config files

Set up the two files from the last two slides in your own repo, then watch Copilot follow them.

- 1 Create copilot-instructions.md with your test rules
- 2 Add a generate-tests.prompt.md prompt file
- 3 Run /generate-tests and check it obeys both

```
your-repo

.github/
├─ copilot-instructions.md
│   # Jest, one behavior per test,
│   # mock network, cover edge cases
├─ prompts/
│   └─ generate-tests.prompt.md
│       # mode: agent
│       # /generate-tests ${file}
```

Custom chat modes and skills

Define a focused persona with its own instructions and a limited set of tools – a specialist you summon instead of a generalist you nudge.

`*.chatmode.md`

A scoped persona

A "QA reviewer" mode that only critiques and hardens tests

`tools`

Limited blast radius

Grant only the tools the task needs – read, search, run tests

`skillsets`

Shared expertise

Reusable instructions and resources the whole team can load

AGENTS.md personas across the repo

AGENTS.md root level – applies to all agents

Mission: help workshop participants produce reliable tests quickly while preserving application behavior.

Priorities:

1. Keep behavior and API contracts unchanged unless requested.
2. Prefer minimal, reviewable patches over large rewrites.
3. Verify with targeted test commands before broader runs.
4. For review tasks, report findings first.

tests/unit/AGENTS.md scoped to unit tests

Focus: calculateDiscount and pure functions.

Run: `npm run test:unit`

Do not modify `src/` files.

tests/api/AGENTS.md scoped to API tests

Focus: checkout pipeline routes.

Attach: `#file:.copilot/context/domain-rules.md`

Run: `npm run test:api`

Agent mode and the coding agent

Beyond autocomplete: Copilot can plan multi-step work, run your tests, read the failures, and iterate until the suite is green.

- **Runs and reads the suite**

Executes tests in the terminal and reacts to real output

- **Assign an issue, get a PR**

The coding agent works async and opens a pull request

- **Self-corrects on failure**

Edits, re-runs, and keeps going until checks pass

- ! **You still own the merge**

Review every diff – autonomy is not a substitute for judgment

MCP servers

The Model Context Protocol lets Copilot reach beyond your code – into browsers, databases, and trackers – through a standard interface.

- Drive a browser to write real E2E tests
- Read a schema to fabricate valid test data
- ! Vet each server – it runs with your access

```
.vscode/mcp.json

{
  "servers": {
    "playwright": {
      "command": "npx",
      "args": ["@playwright/mcp@latest"]
    }
  }
}
```


Config cheat-sheet

Every config file from this section in one place – copy these into your repo after the workshop.

workshop-exercises/.github/copilot-instructions.md

Testing standards

- AAA; one behavior per it().
- Assert exact values, not toBeTruthy.
- Golden path, boundaries, errors.
- Don't mock the module under test.
- No Math.random()/Date.now().
- APIs: status AND envelope; no secrets/PII in fixtures.

.github/prompts/generate-tests.prompt.md

mode: agent

description: Generate tests for a file

Write Jest tests for \${file}.

- Cover happy path, edge, and error cases.
- Reuse factories from tests/factories.
- Do not test private functions.
- Stop and ask if behavior is unclear.

qa-reviewer.chatmode.md

description: QA reviewer – harden tests

tools: [read, search, runTests]

You are a meticulous QA reviewer.

Critique tests for weak assertions and flaky patterns; suggest fixes. Do not write feature code.

.vscode/mcp.json

```
{
  "servers": {
    "playwright": {
      "command": "npx",
      "args": ["@playwright/mcp@latest"]
    }
  }
}
```

Review, validate, and govern

Generated tests are a draft. Here is how a team turns drafts into trustworthy, production-grade coverage.

AI shifts the work — it doesn't remove it

The effort moves up the value chain. Less typing, more thinking.



Writing

Copilot drafts the boilerplate, scaffolding, and first-pass tests.



Reviewing

You validate assertions, catch weak and flaky tests, own the merge.



Designing

You decide what matters: behaviors worth testing, risk, and intent.

Reviewing AI-generated tests

Treat every generated test like a pull request from a junior teammate.

- [1] Does it assert the actual behavior, not an implementation detail?
- [2] Would it fail if the code were broken on purpose?
- [3] Are the inputs realistic and the edges covered?
- [4] Is it isolated, deterministic, and free of hidden state?
- [5] Is the name a clear statement of the behavior under test?
- [6] Does it avoid leaking secrets, tokens, or real user data?

Failure modes to catch

Flaky tests

Pass or fail depending on timing, order, or environment. They erode trust in the whole suite.

False confidence

High counts and green checks that mask shallow assertions and untested branches.

Weak assertions

Checks for truthiness or no error, instead of the specific value or behavior expected.

A quick gut check: would this test still pass if the feature quietly stopped working? If yes, it is not protecting you.

Security considerations

- ! **No real secrets in tests**

Keep keys, tokens, and PII out of fixtures and prompts

- ! **Test the abuse paths**

Auth, input validation, and injection scenarios

- ! **Mind the prompt context**

Know what code and data the assistant can see

- ! **Don't trust copied snippets**

Verify suggested dependencies and license terms

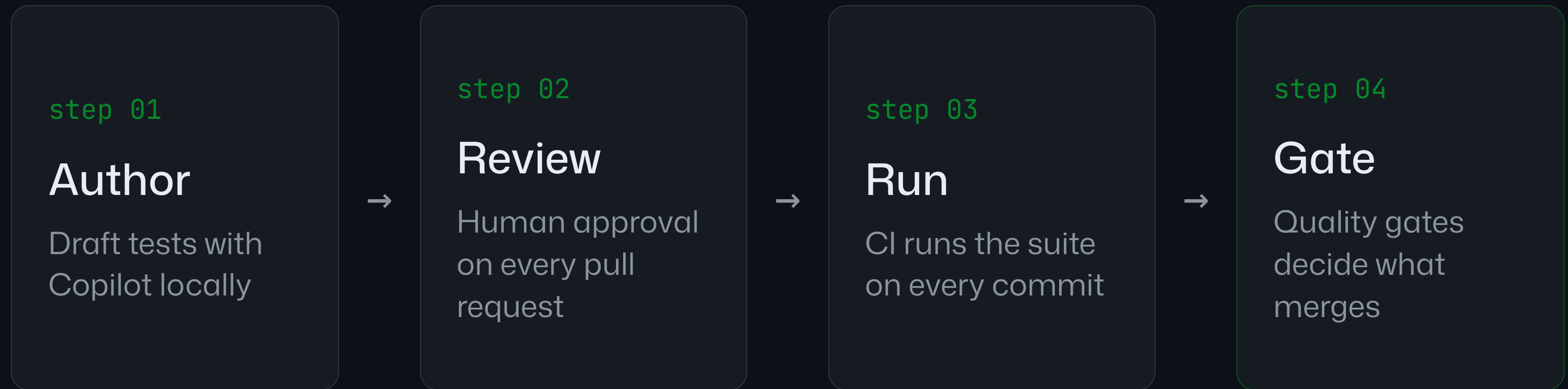
The assistant accelerates the work; your security review still owns the outcome.

PART 04

Pipelines, gates, and adoption

Where AI-assisted tests live in CI/CD, the gates that protect main, and how a team adopts this without chaos.

AI tests in the pipeline



Generation speeds up authoring. Review and gates make sure nothing reaches main without a human and the pipeline agreeing.

// 41 - STANDARDS

Quality gates

Gates make the standard automatic. They block merges that fall short, so trust does not depend on memory or mood.

Tune thresholds to your risk. Coverage is a floor, not the goal.

All tests green	required
Coverage threshold	enforced
No flaky reruns to pass	tracked
Human review approved	required
Security scan clean	blocking

Team-wide adoption

01 Agree on what good looks like

A shared review checklist and test standards

02 Keep humans accountable

Authorship and ownership stay with people

03 Make gates the safety net

Standards enforced in CI, not just in docs

04 Share prompts and patterns

Capture what works so the team levels up together

05 Measure flakiness and escapes

Track defects that slip past the suite

06 Start small, then scale

Pilot on one service before rolling out widely

Guardrails to apply today

- 01 Generate fast, but review every test like production code.
- 02 Assert real behavior – a green suite is not the goal.
- 03 Enforce standards with quality gates, not good intentions.
- 04 Keep a human accountable for what ships.

OPEN FLOOR

Q&A



SCAN TO CLONE THE REPO



SCAN TO GIVE FEEDBACK

THANK YOU

Keep the speed. Keep the trust.

Go build tests your team can rely on – and let Copilot do the heavy lifting.

moataznabil.blog • [LinkedIn](#) • [GitHub](#)